

M1.1: 进程、线程与协程物理调度状态转换

Theory: 操作系统资源分配与调度的基本单元。进程拥有独立的虚拟地址空间；线程是 CPU 调度的最小单位，共享进程资源；协程在用户态运行，由用户级调度器调度，切换时无须进入内核态，避免了高昂的内核上下文切换开销。

Details: 线程上下文切换需保存和恢复 CPU 寄存器（如 PC, SP）和程序计数器，并使 CPU 缓存（TLB）失效。协程则采用非抢占式或协同式多任务调度，通常只需在用户栈上保存几十个字节的局部变量，切换开销比线程低 10-100 倍。

Trade-off: 协程虽然具有极高的并发处理能力，但无法利用多核 CPU。如果在协程内执行阻塞式 I/O（如直接调用阻塞的 `read()`），会导致挂载它的整个物理 OS 线程被挂起，造成饥饿。必须将其绑定在异步非阻塞 I/O（如 `epoll`）上。

M1.3: 进程间通信（IPC）机制与共享内存信号量互斥

Theory: 进程之间由于地址空间绝对隔离，必须通过内核提供的中介通道来进行数据传输或同步。

Details: IPC 包含多种方式：管道（半双工，通过内核环形缓冲区传输，只支持父子进程）、命名管道（FIFO，支持任意进程）、共享内存（最快，将同一物理内存映射到不同进程的虚拟空间，无需数据拷贝，但必须配合信号量 `sem_t` 实现互斥互锁）。

Trade-off: 共享内存读写完全不受内核保护，容易产生严重的竞态条件（Race Condition）。开发中若没有正确使用原子操作（CAS）或互斥信号量对临界区进行加锁，会导致内存数据脏写和进程死锁。

M1.4: 文件系统物理结构：inode 索引节点与虚拟文件系统

Theory: 操作系统对持久化存储设备的抽象与管理。虚拟文件系统（VFS）屏蔽了物理文件系统（如 ext4, xfs, NTFS）的底层差异，通过统一的接口（`read`, `write`）向进程提供标准调用。

Details: 文件系统物理上由 superblock（超级块）、inode 数组和 data blocks 组成。每个文件均有一个唯一的 inode 编号，记录了文件的元数据（权限、大小、修改时间）以及指向实际数据块的指针；文件名与 inode 编号的映射关系则保存在目录文件中。

Trade-off: inode 数量在格式化磁盘时就已经确定。如果小文件过多，即使磁盘还有充足的物理空间，若 inode 号全部耗尽，系统也会报 `No space left on device`。因此，海量小文件必须使用对象存储或合并存储。

M1.2: 虚拟内存、物理内存页表与段页式 MMU 地址翻译

Theory: 虚拟内存为每个进程提供连续、隔离的线性地址空间。操作系统通过内存管理单元（MMU）将虚拟地址映射到不连续的物理内存。

Details: 映射通过多级页表（如 64 位系统下的 4 级页表）进行。虚拟地址（VA）被分解为不同级别的页表索引和物理页内偏移（Offset）。转换后的物理页帧号（PFN）与偏移拼接得到物理地址。为了加速映射，CPU 内部集成了页表缓存（TLB）。

Trade-off: 多级页表虽然大幅减少了页表常驻内存所需的空间，但增加了内存访问的次数（例如 4 级页表在未命中 TLB 时需要 5 次内存访问）。为此，必须优化 TLB 命中率，并在重载内存场景下使用大页（Huge Pages，如 2MB 或 1GB）来降低 TLB 失效率。

M1.5: Socket 网络套接字与五种系统级 I/O 模型

Theory: 描述进程通过 Socket 句柄从网络收发数据包的内核调用机制。

Details: 五种 I/O 模型为：1. 阻塞（进程一直等待直到数据就绪）；2. 非阻塞（系统调用立刻返回 `EWouldBlock`，通过轮询获取就绪状态）；3. I/O 多路复用（通过 `select/poll/epoll` 单线程监听数万个套接字，就绪时通知进程）；4. 信号驱动 I/O；5. 异步 I/O（AIO，完全由内核在后台完成读写并将数据拷贝至用户空间后，再通知进程）。

Trade-off: 异步 I/O 在理论上效率最高，但在 Linux 系统中实现（如 POSIX AIO）并不完善且难以调试。实际高性能网关均采用 `epoll`（边沿触发，ET）结合非阻塞 I/O，并配合 Reactor 事件模式来实现最大吞吐。

M2.1: 关系型数据库事务隔离级别与 MVCC 并发控制

Theory: 数据库事务必须满足 ACID。为了在保证一致性的前提下提升并发性能，现代数据库引入隔离级别以及多版本并发控制（MVCC）。

Details: 隔离级别由低到高为：读未提交、读已提交、可重复读、串行化。MVCC 通过在每行记录中隐式添加事务 ID（DB_TRX_ID）和回滚指针（DB_ROLL_PTR），配合 ReadView（一致性读视图）在 Undo Log 中寻找历史版本，实现读写互不阻塞。

Trade-off: 可重复读隔离级别在特定数据库中（如 InnoDB）能解决幻读问题，但在极高并发下，行锁与间隙锁（Next-Key Lock）的频繁升级会导致死锁（Deadlock）。在高频更新场景下，需尽量减小事务范围以防锁死。

M2.2: Redis 内存缓存系统与 AOF/RDB 持久化

Theory: 极高性能的内存键值对存储数据库。Redis 核心逻辑采用单线程架构，避免了多线程上下文切换和锁竞争开销，依托 I/O 多路复用处理高并发请求。

Details: 持久化机制有两种：RDB（定期将当前内存数据生成快照保存，快照基于 `fork` 子进程和写时复制 - COW 进行，不阻塞主进程）和 AOF（顺序追加写入命令日志，AOF 重写会通过子进程合并命令精简大小）。

Trade-off: Redis 虽然运行速度极快，但在大流量下若发生 AOF 频繁刷盘（`appendfsync always`），会导致严重的 I/O 阻塞。另外，Redis 单线程架构决定了如果遇到大 Key 读写或 `KEYS *` 这类 $O(N)$ 慢命令，会引起整个 Redis 服务停顿故障崩溃。

M2.3: Raft 分布式一致性共识协议与状态机

Theory: 分布式系统为了防止单点故障，通常在多个节点间通过共识协议来维护一个强一致、可复制的状态机（Replicated State Machine）。

Details: Raft 将共识问题分解为三个核心子问题：1. 领导者选举（通过心跳机制与 Term 单调递增周期判断，投票必须满足“过半同意且选民日志新于候选人”）；2. 日志复制（Leader 接收写请求后，分发给 Follower，当多数派节点成功写入后提交 `commitIndex`，并向客户端返回成功）；3. 安全性（保证已提交日志不会被覆盖）。

Trade-off: Raft 在网络分区（Network Partition）时可能会产生脑裂（Split-Brain），即出现两个 Leader。为防止脏读，Leader 必须定期与多数派确认心跳，或者通过 Lease 租约机制保障自身权威期方可响应读取。

M2.4: 消息队列机制：Kafka 高吞吐顺序追加写与稀疏索引

Theory: 解耦分布式系统、削峰填谷的核心组件。Kafka 以极高吞吐量著称，其核心原理是把磁盘当作内存使用，依靠操作系统 Page Cache 与顺序追加写（Append-only Log）大幅提升磁盘 I/O 吞吐。

Details: Kafka 消息以 Partition 为单位按物理文件分片（Segment）。每个 Segment 包含 `.log`（消息文件）与 `.index`（稀疏索引文件）。查找时利用二分法在稀疏索引中快速定位消息在文件中的物理偏移量，并通过 `send-file` 零拷贝技术直接发送到网络。

Trade-off: 零拷贝和顺序写带来极高吞吐，但在极端停电故障下，Page Cache 中尚未刷入磁盘的数据会发生静默丢失。为了保障金融级数据零丢失，必须配置 `acks=all` 并启用多副本同步确认（ISR）。

M2.5: 微服务服务治理与 gRPC 二进制 Protobuf 编码

Theory: 在分布式微服务架构中，服务实例频繁扩缩容。服务治理负责实现服务的注册、发现、健康监测以及服务间的高效通信。

Details: gRPC 是基于 HTTP/2 的高性能 RPC 框架。它利用二进制格式的 Protocol Buffers (Protobuf) 来定义接口与数据模型。Protobuf 使用 Varint (可变长度整型) 和 Tag-Length-Value (TLV) 编码，比 JSON/XML 的体积小 3-10 倍。

Trade-off: 二进制序列化显著减少了带宽占用和 CPU 解析时间，但其最大劣势是数据可读性极差，无法通过简单的网络抓包包流直接阅读，在生产调试和网关转发层会增加排查成本。

M3.1: 现代浏览器渲染管线：从 DOM 构建到 Composite 合成

Theory: 浏览器将 HTML/CSS/JS 代码转换为屏幕物理像素的流水线。

Details: 管线步骤: 1. 解析 HTML 生成 DOM 树, 解析 CSS 生成 CSSOM 树; 2. 结合生成 Attachment Layout 布局树, 计算每个元素的坐标和大小; 3. Paint (绘制), 将布局树拆分为不同的物理图层 (Layers) 并生成绘制指令; 4. Rasterize (光栅化), 将图层指令转换为位图物理像素; 5. Composite (合成), 利用 GPU 将位图图层合成并输出到显示器。

Trade-off: 频繁触发 Layout (重排/Reflow) 或 Paint (重绘/Repaint) 会极大消耗 CPU 资源。优化前端页面渲染性能时, 必须利用 CSS3 的 'transform' 或 'opacity' 动画, 绕过重排与重绘, 直接在 GPU 触发 Composite 合成渲染, 防止卡顿。

M3.3: 构建与工程化工具：Webpack 依赖图与 Vite 极速启动

Theory: 前端代码需要经过转译、打包、合并和压缩后才能被浏览器加载。构建工具负责管理复杂的资源依赖链条。

Details: Webpack 采用静态构建模式，从入口文件递归解析依赖，在内存中构建完整的模块依赖图 (Module Graph)，然后将它们融合成打包后的 Chunk 块。Vite 采用现代浏览器原生 ESM 的动态加载模式，在本地开发时不进行打包，而是通过 ESBuild 极速转译请求的模块，实现秒级冷启动。

Trade-off: Webpack 生产构建极慢, 但具有强大的打包和插件生态; Vite 在本地冷启动极快, 但在生产构建时仍需调用 Rollup 进行打包, 且由于本地依赖海量 HTTP 请求, 如果模块过多, 会带来严重的网络瀑布流加载迟滞。

M3.5: Web 安全防护规范：XSS、CSRF 与 CORS

Theory: 浏览器同源策略 (Same-Origin Policy) 限制了不同域名间的网络交互。攻击者通过利用网页漏洞来窃取用户凭证或越权操作。

Details: XSS (跨站脚本攻击) 指注入恶意脚本执行，防御手段是严格执行 HTML 转义与 CSP (内容安全策略)；CSRF (跨站请求伪造) 指利用用户 Cookie 凭证发起未授权请求，防御手段是引入 CSRF Token 或使用 Cookie 'SameSite' 属性；CORS (跨域资源共享) 是浏览器的一种安全控制，允许服务端通过返回响应头 'Access-Control-Allow-Origin' 来控制跨域读写权限。

Trade-off: 开启通配符跨域配置 'Access-Control-Allow-Origin: *' 会导致同源保护防线彻底消失，允许任意恶意网站读取用户的敏感接口数据，必须严格使用动态安全域名校验。

M3.2: 前端虚拟 DOM 与 Reconciliation 协调过程及 Diff 算法

Theory: 减少频繁操作真实 DOM 带来的巨额浏览器渲染树重建开销。通过在内存中用轻量 JS 对象来模拟 DOM 树，在状态变更时，利用算法对比新旧两棵树的差异，实现局部按需更新。

Details: 协调过程 (Reconciliation) 中，React/Vue 使用同层对比的启发式 Diff 算法，将复杂度从传统树 Diff 的 $O(N^3)$ 降至 $O(N)$ 。算法假设：若节点类型不同，直接销毁并重新创建该树枝；对于列表节点，通过唯一的 'key' 属性来识别元素是否被移动、插入或删除。

Trade-off: 在动态列表渲染中，若使用数组索引 'index' 作为 'key'，当在数组头部插入元素时，会导致 Diff 算法误判所有后续列表项均发生了变化，从而触发全量重新渲染，造成严重的性能衰退。

M3.4: 单页应用 (SPA) 前端路由与全局状态管理

Theory: 单页应用 (SPA) 只在初始化时加载一个 HTML 模板。前端路由负责在不刷新页面的前提下，通过劫持浏览器 URL 的变化来切换不同的组件视图；全局状态管理则负责维护多个不相邻组件间的共享状态。

Details: 路由实现主要有 Hash 路由 (利用 'window.onhashchange') 与 History 路由 (利用 HTML5 的 'history.pushState' 且不发起网络请求)。状态管理 (如 Redux) 采用单向数据流 (Action Reducer Store)，保证状态流转的可追溯与可调试性。

Trade-off: 强行把所有组件的私有状态都放入全局状态管理 (如 Store) 会导致全局状态极度臃肿，且每一次状态更新都会触发无关组件的重复渲染。应区分全局共享状态与组件私有状态。

M4.1: Linux Namespace 命名空间与 Cgroups 资源隔离制

Theory: 现代容器技术 (如 Docker) 的基础沙盒机制。容器并非虚拟机，而是运行在宿主机内核之上的特殊物理进程。

Details: Linux Namespace 负责进程运行环境的逻辑隔离，包括 PID (进程号)、NET (网络栈)、IPC (信号量)、MNT (挂载文件点) 及 UTS (主机名)。Cgroups (控制组) 则负责物理资源的硬限制，能硬性限制该容器进程可分配的 CPU 占比、内存配额 (Memory Limit) 与 I/O 带宽。

Trade-off: 容器共享宿主机内核。一旦容器进程通过系统调用漏洞攻破内核屏障，将直接威胁宿主机安全；此外，如果 Cgroups 内存配额限制设置不合理，当容器内存耗尽时，会触发 Linux 内核的 OOM Killer，容器进程会被强行终止，引发应用停机。

M4.2: 容器编排架构: Kubernetes 核心控制器声明式 API

Theory: 大规模容器集群的管理引擎。Kubernetes 放弃了命令式操作, 全面采用声明式 (Declarative) API, 通过定义“期望状态”, 由后台控制器持续运行调解环路 (Reconciliation Loop) 去逼近并维持该状态。

Details: 核心控制器 (如 Deployment) 通过 ReplicaSet 管理 Pod 实例。Deployment 控制器监听集群状态; 如果实际 Pod 数少于期望数, 则通知 Scheduler 在空闲 Node 上调度运行新 Pod; 如果某 Pod 物理节点 Crash, 则自动将其漂移并重新创建。

Trade-off: 声明式 API 带来了强大的自治与自愈能力, 但带来了极高的控制流延迟与系统复杂性。如果控制循环出现配置错误 (如滚动更新时就绪指标设置过宽), 会导致故障实例被全部滚动上线, 引发大面积雪崩。

M4.3: CI/CD 自动化管线: Docker 构建缓存优化与发布策略

Theory: 自动化代码提交流程至生产发布。CI/CD 管线利用自动化测试、打包并最终推送到生产环境。

Details: 优化 CI/CD 速度的核心在于利用 Docker 构建分层缓存 (Layer Cache): 在编写 Dockerfile 时, 应将不经常变动的依赖安装 (如 'npm install') 放在经常变动的源码复制 ('COPY', ':') 之前。发布策略包括蓝绿部署 (同时维护两套完全一样的环境, 切换网关流量) 和金丝雀/灰度发布 (逐步将小比例流量分流至新版本进行验证)。

Trade-off: 蓝绿部署需要双倍的服务器硬件资源开销, 而灰度发布则可能导致用户在一个 Session 中同时访问到新旧两个版本的系统, 需配合 Session 粘性路由或做向下兼容。

M4.4: Kubernetes 存活探针 (Liveness) 与就绪探针 (Readiness)

Theory: 容器级别的健康检查探针。Liveness 用于判断容器进程是否存活 (不存活则强行重启); Readiness 用于判断容器是否已经初始化完成, 可以接收网关流量。

Details: 探针检测机制包括 HTTP 请求、TCP 端口探测或在容器内执行 Exec 命令。如果 Readiness 探针连续失败, Service 控制器会将该 Pod 的 IP 从负载均衡端点 (Endpoints) 中剔除, 暂停向其分发请求。

Trade-off: 混淆两者的职责是生产灾难根源。如果将 Liveness 探针指向依赖下游数据库的健康检测接口, 一旦数据库暂时不可用, 会导致集群内所有 Pod 都在同一时刻被 Liveness 强制不停重启, 加剧整体雪崩。

M5.1: 一致性哈希 (Consistent Hashing) 算法负载均衡机制

Theory: 解决传统哈希算法 ('hash(key) % N') 在服务器节点增加或减少时导致全量缓存失效的缺陷。

Details: 将哈希值空间映射为一个 $0 \sim 2^{32} - 1$ 的虚拟闭环。服务器节点通过其 IP 或名称进行哈希, 顺时针排列在哈希环上。数据 Key 经哈希后, 沿环顺时针寻找遇到的第一台物理服务器进行存储。为了解决由于节点分布不均带来的负载倾斜, 引入虚拟节点 (Virtual Nodes) 技术。

Trade-off: 当某节点突发下线时, 其顺时针方向的紧邻节点将承载其全部流量, 容易引发级联雪崩。因此, 虚拟节点的哈希环设计必须保证哈希分布的平滑性, 并预留充足的冗余容量。

M5.2: 高并发系统降级与熔断防护机制

Theory: 在大流量下, 当某个下游非核心依赖服务 (如评分服务) 发生延迟响应或频繁报错时, 必须快速隔离该依赖, 防止其占满系统线程资源导致整个微服务群雪崩。

Details: 熔断状态机包含三种状态: Closed (正常, 流量全通)、Open (发生异常, 当错误率或慢调用比例在滑窗内达到阈值时开启, 快速返回本地失败 Fallback)、Half-Open (尝试恢复, 经过冷却期后允许极小流量尝试请求, 若全部成功则恢复到 Closed, 若报错则重新退回到 Open)。

Trade-off: 降级熔断属于有损防御。一旦开启熔断, 用户在该链路上的体验会退化 (例如无法看到商品评论)。熔断阈值的配置必须基于压测历史, 避免在正常波动期间发生系统“防御过载”。

M5.3: 系统限流算法: 漏桶算法与令牌桶算法

Theory: 控制请求流量进入服务器的速率, 防止过载请求冲垮服务器。

Details: 漏桶 (Leaky Bucket) 算法: 水 (请求) 以任意速率注入桶中, 桶底部以恒定速率漏水, 当注水速率过快、桶满时, 溢出的请求直接丢弃。令牌桶 (Token Bucket) 算法: 系统以恒定速率向桶中放入令牌, 当请求到达时必须从桶中消耗令牌方可处理, 当桶中无令牌时, 多余请求被限流拦截。

Trade-off: 漏桶算法强制输出恒定速率, 无法应对高并发系统中最常见的突发流量 (Burst traffic); 令牌桶由于允许在空闲时囤积令牌, 能完美支持瞬时的突发流量。应根据业务场景 (如网关限流选令牌桶, 消息流处理选漏桶) 进行选型。

M5.4: 数据库水平拆分、Shard Key 路由与 Snowflake ID

Theory: 当单表数据量过大 (如超过两千万行) 时, 数据库的 B+ 树索引层数过深, 导致查询性能暴跌, 必须将数据水平拆分到多台物理机器上。

Details: 水平拆分的核心在于选择合适的 Shard Key (分片键, 如用户 ID)。所有读写请求均根据 Shard Key 哈希后路由到特定的物理数据库。分布式下由于无法使用自增主键, 必须使用全局唯一 ID 生成器, 如 Snowflake (雪花算法), 生成包含时间戳 (41 位)、工作节点 ID (10 位) 和序列号 (12 位) 的 64 位趋势递增整数。

Trade-off: 水平拆分后, 若查询条件中没有携带 Shard Key, 查询路由会退化为“全库广播扫描”, 并发吞吐降为冰点。此外, 跨分片 (Cross-shard) 的 Join 操作和分布式事务执行成本极高, 应通过数据冗余和最终一致性 (Saga) 规避。

M5.5: 系统可监控性 (Observability) 三位一体: Metric, Log & Trace

Theory: 描述生产环境下分布式系统的内部状态。可监控性通过指标、日志和追踪来实现系统异常的快速排查。

Details: 指标 (Metric): 时序聚合数据, 描述系统负荷 (如 CPU、QPS), 占用内存小, 用于报警; 日志 (Log): 针对特定时间点的详细非结构化事件描述, 用于细粒度排查; 追踪 (Trace): 记录一个分布式请求跨越多个微服务的完整包流调用链路, 利用全局 'TraceID' 与 'SpanID' 实现全链路时延追踪。

Trade-off: 海量的 Trace 链路追踪和详尽的 Log 会带来巨大的网络带宽和分布式存储开销 (甚至超过业务系统本身的服务器开销)。在生产实践中, 必须引入采样率控制 (如只采样 1% 的 Trace), 降低非业务开销。

M6.1: 领域驱动设计（DDD）战略限界上下文与战术聚合根

Theory: 解决大型复杂业务系统设计中，软件复杂度随时间呈指数级上升的规范论。

Details: 战略设计：通过划定“限界上下文（Bounded Context）”来确立概念的语义边界，不同的上下文之间通过防腐层（ACL）交互，防止业务概念被污染。战术设计：在上下文内部，通过“聚合根（Aggregate Root）”对外暴露，统一控制聚合内部实体（Entities）和值对象（Value Objects）的状态变更，严格维持领域内的事务与数据一致性边界。

Trade-off: DDD 拥有极强的系统解耦和扩展性优势，但学习曲线极陡，前期建模开销巨大。如果在一个简单的增删改查（CRUD）管理后台中强行引入 DDD 复杂的战术模型，会造成严重的多层冗余代码堆砌。

M6.3: 无服务器架构（Serverless）冷启动优化

Theory: 开发者只需编写并提交业务函数，无需管理底层服务器实例。基础设施会根据并发请求自动进行极速的扩缩容，当无流量时，服务器实例直接缩容至零。

Details: 当流量在缩容至零后突然涌入，平台需要经历虚拟机/容器创建、网络分配和运行时初始化，这一过程产生的响应延迟称为“冷启动”。优化冷启动的主要手段包括：使用轻量化运行时（如 Node.js 代替 Java VM）、代码打包体积最小化压缩（Tree Shaking）、运行时预热与利用轻量级沙箱（如 Firecracker MicroVM）加速启动。

Trade-off: Serverless 大幅节约了空闲服务器开销，但不适合对首包响应时间（First Byte Delay）极其敏感的强实时在线系统。对于这类场景，必须预留极小比率的“预热/常驻实例”，这会产生基础折中成本。

M6.4: LLMOps 运维与向量数据库 RAG 检索拓朴

Theory: 在大模型应用交付工程（LLMOps）中，由于 LLM 存在时效性短和幻觉问题，通常引入检索增强生成（RAG），利用外挂向量数据库注入知识。

Details: RAG 拓扑流：1. 离线阶段：将企业文档切割为文本块（Chunking），通过嵌入模型（Embedding Model）转换为高维实数向量，存入向量数据库中构建索引（如 HNSW）；2. 在线阶段：用户发起提问，提问经实时量化后，在向量数据库中检索余弦相似度（Cosine Similarity）最高的前 K 个文档块，将它们作为上下文（Context）拼接入提示词（Prompt）发送给大模型，生成事实准确的回答。

Trade-off: RAG 的性能极其依赖文本分块的切分策略和向量相似度检索精度。如果切分时截断了上下文语义，或者由于向量空间碰撞引入了噪音文档块，反而会误导大模型，使其产生更加不可控的幻觉。

M6.2: 测试驱动开发（TDD）红-绿-重构环路规范

Theory: 一种“测试先行”的系统工程开发方法论。通过编写测试来理清并定义代码的行为边界，依靠持续重构保障代码结构的可读性与健康度。

Details: TDD 的开发周期分为三步：1. 红色阶段：根据业务功能要求，先写单元测试，并执行测试（此时无业务实现代码，测试必然报错报错）；2. 绿色阶段：编写能让该测试通过的最简业务实现代码，使测试成功（亮绿灯）；3. 重构阶段：在测试绿灯的保护屏障下，重构优化冗余代码，并保证重构后测试依然亮绿灯。

Trade-off: TDD 能极大提升代码测试覆盖率和设计质量。然而，频繁的开发环路转换对开发人员素质要求极高，如果需求频繁发生剧烈变动，会导致测试用例的维护成本成倍增加，应平衡核心业务覆盖面。

Zone T1: Full-Stack Core APIs

os.fork(), select.epoll(), react.createContext(), webpack.compile(), docker.run(), k8s.v1.Pod(), express.Router(), grpc.dial()

Zone T2: System Faults & Protections

out_of_memory_killer, thundering_herd, cors_policy_violation, xss_injection, deadlock, pod_crash_loop, split_brain